

# Atividade 02 Árvores de Decisão e Florestas Aleatórias

March 31, 2026

```
[1]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris, load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
```

## 1 Parte 1

```
[2]: # Carregando o dataset
iris = load_iris()
iris
```

```
[2]: {'data': array([[5.1, 3.5, 1.4, 0.2],
[4.9, 3. , 1.4, 0.2],
[4.7, 3.2, 1.3, 0.2],
[4.6, 3.1, 1.5, 0.2],
[5. , 3.6, 1.4, 0.2],
[5.4, 3.9, 1.7, 0.4],
[4.6, 3.4, 1.4, 0.3],
[5. , 3.4, 1.5, 0.2],
[4.4, 2.9, 1.4, 0.2],
[4.9, 3.1, 1.5, 0.1],
[5.4, 3.7, 1.5, 0.2],
[4.8, 3.4, 1.6, 0.2],
[4.8, 3. , 1.4, 0.1],
[4.3, 3. , 1.1, 0.1],
[5.8, 4. , 1.2, 0.2],
[5.7, 4.4, 1.5, 0.4],
[5.4, 3.9, 1.3, 0.4],
[5.1, 3.5, 1.4, 0.3],
[5.7, 3.8, 1.7, 0.3],
[5.1, 3.8, 1.5, 0.3],
[5.4, 3.4, 1.7, 0.2],
[5.1, 3.7, 1.5, 0.4],
```

[4.6, 3.6, 1. , 0.2],  
[5.1, 3.3, 1.7, 0.5],  
[4.8, 3.4, 1.9, 0.2],  
[5. , 3. , 1.6, 0.2],  
[5. , 3.4, 1.6, 0.4],  
[5.2, 3.5, 1.5, 0.2],  
[5.2, 3.4, 1.4, 0.2],  
[4.7, 3.2, 1.6, 0.2],  
[4.8, 3.1, 1.6, 0.2],  
[5.4, 3.4, 1.5, 0.4],  
[5.2, 4.1, 1.5, 0.1],  
[5.5, 4.2, 1.4, 0.2],  
[4.9, 3.1, 1.5, 0.2],  
[5. , 3.2, 1.2, 0.2],  
[5.5, 3.5, 1.3, 0.2],  
[4.9, 3.6, 1.4, 0.1],  
[4.4, 3. , 1.3, 0.2],  
[5.1, 3.4, 1.5, 0.2],  
[5. , 3.5, 1.3, 0.3],  
[4.5, 2.3, 1.3, 0.3],  
[4.4, 3.2, 1.3, 0.2],  
[5. , 3.5, 1.6, 0.6],  
[5.1, 3.8, 1.9, 0.4],  
[4.8, 3. , 1.4, 0.3],  
[5.1, 3.8, 1.6, 0.2],  
[4.6, 3.2, 1.4, 0.2],  
[5.3, 3.7, 1.5, 0.2],  
[5. , 3.3, 1.4, 0.2],  
[7. , 3.2, 4.7, 1.4],  
[6.4, 3.2, 4.5, 1.5],  
[6.9, 3.1, 4.9, 1.5],  
[5.5, 2.3, 4. , 1.3],  
[6.5, 2.8, 4.6, 1.5],  
[5.7, 2.8, 4.5, 1.3],  
[6.3, 3.3, 4.7, 1.6],  
[4.9, 2.4, 3.3, 1. ],  
[6.6, 2.9, 4.6, 1.3],  
[5.2, 2.7, 3.9, 1.4],  
[5. , 2. , 3.5, 1. ],  
[5.9, 3. , 4.2, 1.5],  
[6. , 2.2, 4. , 1. ],  
[6.1, 2.9, 4.7, 1.4],  
[5.6, 2.9, 3.6, 1.3],  
[6.7, 3.1, 4.4, 1.4],  
[5.6, 3. , 4.5, 1.5],  
[5.8, 2.7, 4.1, 1. ],  
[6.2, 2.2, 4.5, 1.5],

[5.6, 2.5, 3.9, 1.1],  
 [5.9, 3.2, 4.8, 1.8],  
 [6.1, 2.8, 4. , 1.3],  
 [6.3, 2.5, 4.9, 1.5],  
 [6.1, 2.8, 4.7, 1.2],  
 [6.4, 2.9, 4.3, 1.3],  
 [6.6, 3. , 4.4, 1.4],  
 [6.8, 2.8, 4.8, 1.4],  
 [6.7, 3. , 5. , 1.7],  
 [6. , 2.9, 4.5, 1.5],  
 [5.7, 2.6, 3.5, 1. ],  
 [5.5, 2.4, 3.8, 1.1],  
 [5.5, 2.4, 3.7, 1. ],  
 [5.8, 2.7, 3.9, 1.2],  
 [6. , 2.7, 5.1, 1.6],  
 [5.4, 3. , 4.5, 1.5],  
 [6. , 3.4, 4.5, 1.6],  
 [6.7, 3.1, 4.7, 1.5],  
 [6.3, 2.3, 4.4, 1.3],  
 [5.6, 3. , 4.1, 1.3],  
 [5.5, 2.5, 4. , 1.3],  
 [5.5, 2.6, 4.4, 1.2],  
 [6.1, 3. , 4.6, 1.4],  
 [5.8, 2.6, 4. , 1.2],  
 [5. , 2.3, 3.3, 1. ],  
 [5.6, 2.7, 4.2, 1.3],  
 [5.7, 3. , 4.2, 1.2],  
 [5.7, 2.9, 4.2, 1.3],  
 [6.2, 2.9, 4.3, 1.3],  
 [5.1, 2.5, 3. , 1.1],  
 [5.7, 2.8, 4.1, 1.3],  
 [6.3, 3.3, 6. , 2.5],  
 [5.8, 2.7, 5.1, 1.9],  
 [7.1, 3. , 5.9, 2.1],  
 [6.3, 2.9, 5.6, 1.8],  
 [6.5, 3. , 5.8, 2.2],  
 [7.6, 3. , 6.6, 2.1],  
 [4.9, 2.5, 4.5, 1.7],  
 [7.3, 2.9, 6.3, 1.8],  
 [6.7, 2.5, 5.8, 1.8],  
 [7.2, 3.6, 6.1, 2.5],  
 [6.5, 3.2, 5.1, 2. ],  
 [6.4, 2.7, 5.3, 1.9],  
 [6.8, 3. , 5.5, 2.1],  
 [5.7, 2.5, 5. , 2. ],  
 [5.8, 2.8, 5.1, 2.4],  
 [6.4, 3.2, 5.3, 2.3],

```

[6.5, 3. , 5.5, 1.8],
[7.7, 3.8, 6.7, 2.2],
[7.7, 2.6, 6.9, 2.3],
[6. , 2.2, 5. , 1.5],
[6.9, 3.2, 5.7, 2.3],
[5.6, 2.8, 4.9, 2. ],
[7.7, 2.8, 6.7, 2. ],
[6.3, 2.7, 4.9, 1.8],
[6.7, 3.3, 5.7, 2.1],
[7.2, 3.2, 6. , 1.8],
[6.2, 2.8, 4.8, 1.8],
[6.1, 3. , 4.9, 1.8],
[6.4, 2.8, 5.6, 2.1],
[7.2, 3. , 5.8, 1.6],
[7.4, 2.8, 6.1, 1.9],
[7.9, 3.8, 6.4, 2. ],
[6.4, 2.8, 5.6, 2.2],
[6.3, 2.8, 5.1, 1.5],
[6.1, 2.6, 5.6, 1.4],
[7.7, 3. , 6.1, 2.3],
[6.3, 3.4, 5.6, 2.4],
[6.4, 3.1, 5.5, 1.8],
[6. , 3. , 4.8, 1.8],
[6.9, 3.1, 5.4, 2.1],
[6.7, 3.1, 5.6, 2.4],
[6.9, 3.1, 5.1, 2.3],
[5.8, 2.7, 5.1, 1.9],
[6.8, 3.2, 5.9, 2.3],
[6.7, 3.3, 5.7, 2.5],
[6.7, 3. , 5.2, 2.3],
[6.3, 2.5, 5. , 1.9],
[6.5, 3. , 5.2, 2. ],
[6.2, 3.4, 5.4, 2.3],
[5.9, 3. , 5.1, 1.8]]),
'target': array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2]),
'frame': None,
'target_names': array(['setosa', 'versicolor', 'virginica'], dtype='<U10'),
'DESCR': '.. _iris_dataset:\n\nIris plants
dataset\n-----\n\n**Data Set Characteristics:**\n\nNumber of
Instances: 150 (50 in each of three classes)\nNumber of Attributes: 4 numeric,

```

```

predictive attributes and the class\n:Attribute Information:\n    - sepal length
in cm\n    - sepal width in cm\n    - petal length in cm\n    - petal width in
cm\n    - class:\n        - Iris-Setosa\n        - Iris-Versicolour\n
- Iris-Virginica\n\nSummary Statistics:\n\n=====
===== \n
Min Max Mean SD Class
Correlation\n===== \n\nsepal
length: 4.3 7.9 5.84 0.83 0.7826\nsepal width: 2.0 4.4 3.05
0.43 -0.4194\npetal length: 1.0 6.9 3.76 1.76 0.9490 (high!)\npetal
width: 0.1 2.5 1.20 0.76 0.9565 (high!)\n=====
===== \n\nMissing Attribute Values: None\nClass
Distribution: 33.3% for each of 3 classes.\nCreator: R.A. Fisher\nDonor:
Michael Marshall (MARSHALL%PLU@io.arc.nasa.gov)\nDate: July, 1988\n\nThe famous
Iris database, first used by Sir R.A. Fisher. The dataset is taken\nfrom
Fisher\'s paper. Note that it\'s the same as in R, but not as in the
UCI\nMachine Learning Repository, which has two wrong data points.\n\nThis is
perhaps the best known database to be found in the\npattern recognition
literature. Fisher\'s paper is a classic in the field and\nis referenced
frequently to this day. (See Duda & Hart, for example.) The\ndata set contains
3 classes of 50 instances each, where each class refers to a\ntype of iris
plant. One class is linearly separable from the other 2; the\nlatter are NOT
linearly separable from each other.\n\n.. dropdown:: References\n\n - Fisher,
R.A. "The use of multiple measurements in taxonomic problems"\n    Annual
Eugenics, 7, Part II, 179-188 (1936); also in "Contributions to\n
Mathematical Statistics" (John Wiley, NY, 1950).\n - Duda, R.O., & Hart, P.E.
(1973) Pattern Classification and Scene Analysis.\n    (Q327.D83) John Wiley &
Sons. ISBN 0-471-22361-1. See page 218.\n - Dasarathy, B.V. (1980) "Nosing
Around the Neighborhood: A New System\n    Structure and Classification Rule for
Recognition in Partially Exposed\n    Environments". IEEE Transactions on
Pattern Analysis and Machine\n    Intelligence, Vol. PAMI-2, No. 1, 67-71.\n -
Gates, G.W. (1972) "The Reduced Nearest Neighbor Rule". IEEE Transactions\n
on Information Theory, May 1972, 431-433.\n - See also: 1988 MLC Proceedings,
54-64. Cheeseman et al\'s AUTOCLASS II\n    conceptual clustering system finds 3
classes in the data.\n - Many, many more ...'\n',
'feature_names': ['sepal length (cm)',
'sepal width (cm)',
'petal length (cm)',
'petal width (cm)'],
'filename': 'iris.csv',
'data_module': 'sklearn.datasets.data'}

```

```
[3]: iris.feature_names
```

```
[3]: ['sepal length (cm)',
'sepal width (cm)',
'petal length (cm)',
'petal width (cm)']
```

```
[4]: X = pd.DataFrame(iris.data, columns=iris.feature_names)
X
```

```
[4]:      sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
0           5.1           3.5           1.4           0.2
1           4.9           3.0           1.4           0.2
2           4.7           3.2           1.3           0.2
3           4.6           3.1           1.5           0.2
4           5.0           3.6           1.4           0.2
..          ...          ...          ...          ...
145          6.7           3.0           5.2           2.3
146          6.3           2.5           5.0           1.9
147          6.5           3.0           5.2           2.0
148          6.2           3.4           5.4           2.3
149          5.9           3.0           5.1           1.8

[150 rows x 4 columns]
```

```
[5]: y = pd.Series(iris.target)
y
```

```
[5]: 0      0
1      0
2      0
3      0
4      0
..
145    2
146    2
147    2
148    2
149    2
Length: 150, dtype: int64
```

```
[6]: # Dando nomes mais amigáveis às classes (0: Maligno, 1: Benigno)
y = y.map({0: 'Setosa', 1: 'Versicolor', 2: 'Virginica'})
y
```

```
[6]: 0      Setosa
1      Setosa
2      Setosa
3      Setosa
4      Setosa
..
145    Virginica
146    Virginica
147    Virginica
```

```

148     Virginica
149     Virginica
Length: 150, dtype: object

```

```

[7]: # Vamos separar nossos dados em dois conjuntos:
# - TREINO: para ensinar o modelo (70% dos dados)
# - TESTE: para avaliar o modelo em dados que ele nunca viu (30% dos dados)
# `random_state=42` garante que a separação seja sempre a mesma, para
↳reprodutibilidade.
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
↳random_state=42)

```

```

[8]: X_train

```

```

[8]:      sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
81                5.5                2.4                3.7                1.0
133               6.3                2.8                5.1                1.5
137               6.4                3.1                5.5                1.8
75                6.6                3.0                4.4                1.4
109               7.2                3.6                6.1                2.5
..              ...                ...                ...                ...
71                6.1                2.8                4.0                1.3
106               4.9                2.5                4.5                1.7
14                5.8                4.0                1.2                0.2
92                5.8                2.6                4.0                1.2
102               7.1                3.0                5.9                2.1

```

[105 rows x 4 columns]

```

[9]: X_test

```

```

[9]:      sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
73                6.1                2.8                4.7                1.2
18                5.7                3.8                1.7                0.3
118               7.7                2.6                6.9                2.3
78                6.0                2.9                4.5                1.5
76                6.8                2.8                4.8                1.4
31                5.4                3.4                1.5                0.4
64                5.6                2.9                3.6                1.3
141               6.9                3.1                5.1                2.3
68                6.2                2.2                4.5                1.5
82                5.8                2.7                3.9                1.2
110               6.5                3.2                5.1                2.0
12                4.8                3.0                1.4                0.1
36                5.5                3.5                1.3                0.2
9                 4.9                3.1                1.5                0.1
19                5.1                3.8                1.5                0.3

```

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 56  | 6.3 | 3.3 | 4.7 | 1.6 |
| 104 | 6.5 | 3.0 | 5.8 | 2.2 |
| 69  | 5.6 | 2.5 | 3.9 | 1.1 |
| 55  | 5.7 | 2.8 | 4.5 | 1.3 |
| 132 | 6.4 | 2.8 | 5.6 | 2.2 |
| 29  | 4.7 | 3.2 | 1.6 | 0.2 |
| 127 | 6.1 | 3.0 | 4.9 | 1.8 |
| 26  | 5.0 | 3.4 | 1.6 | 0.4 |
| 128 | 6.4 | 2.8 | 5.6 | 2.1 |
| 131 | 7.9 | 3.8 | 6.4 | 2.0 |
| 145 | 6.7 | 3.0 | 5.2 | 2.3 |
| 108 | 6.7 | 2.5 | 5.8 | 1.8 |
| 143 | 6.8 | 3.2 | 5.9 | 2.3 |
| 45  | 4.8 | 3.0 | 1.4 | 0.3 |
| 30  | 4.8 | 3.1 | 1.6 | 0.2 |
| 22  | 4.6 | 3.6 | 1.0 | 0.2 |
| 15  | 5.7 | 4.4 | 1.5 | 0.4 |
| 65  | 6.7 | 3.1 | 4.4 | 1.4 |
| 11  | 4.8 | 3.4 | 1.6 | 0.2 |
| 42  | 4.4 | 3.2 | 1.3 | 0.2 |
| 146 | 6.3 | 2.5 | 5.0 | 1.9 |
| 51  | 6.4 | 3.2 | 4.5 | 1.5 |
| 27  | 5.2 | 3.5 | 1.5 | 0.2 |
| 4   | 5.0 | 3.6 | 1.4 | 0.2 |
| 32  | 5.2 | 4.1 | 1.5 | 0.1 |
| 142 | 5.8 | 2.7 | 5.1 | 1.9 |
| 85  | 6.0 | 3.4 | 4.5 | 1.6 |
| 86  | 6.7 | 3.1 | 4.7 | 1.5 |
| 16  | 5.4 | 3.9 | 1.3 | 0.4 |
| 10  | 5.4 | 3.7 | 1.5 | 0.2 |

```
[10]: print(f"\nDados carregados e divididos:")
      print(f" - Tamanho do conjunto de treino: {len(X_train)} amostras")
      print(f" - Tamanho do conjunto de teste: {len(X_test)} amostras")
```

Dados carregados e divididos:

- Tamanho do conjunto de treino: 105 amostras
- Tamanho do conjunto de teste: 45 amostras

```
[11]: # Criando o modelo da árvore
      arvore = DecisionTreeClassifier(random_state=42)

      # Treinando o modelo com os dados de TREINO
      arvore.fit(X_train, y_train)
```

```
[11]: DecisionTreeClassifier(random_state=42)
```



```
[12]: # Fazendo previsões nos dados de TREINO e de TESTE
pred_treino_arvore = arvore.predict(X_train)
pred_teste_arvore = arvore.predict(X_test)
```

```
[13]: # Avaliando a performance
acc_treino_arvore = accuracy_score(y_train, pred_treino_arvore)
acc_teste_arvore = accuracy_score(y_test, pred_teste_arvore)
```

```
[14]: acc_treino_arvore
```

```
[14]: 1.0
```

```
[15]: acc_teste_arvore
```

```
[15]: 1.0
```

```
[16]: def classificacao(acc):
      if acc == 1.0:
          return "Perfeito!"
      elif acc >= 0.9:
          return "Excelente!"
      elif acc >= 0.8:
          return "Bom!"
      elif acc >= 0.7:
          return "Razoável."
      else:
          return "Ruim."
```

```
[17]: print(f"Acurácia no TREINO: {acc_treino_arvore:.4f}␣
      ↪({classificacao(acc_treino_arvore)})")
print(f"Acurácia no TESTE: {acc_teste_arvore:.4f}␣
      ↪({classificacao(acc_teste_arvore)})")
```

Acurácia no TREINO: 1.0000 (Perfeito!)

Acurácia no TESTE: 1.0000 (Perfeito!)

```
[18]: # Agora, vamos treinar uma floresta com 100 árvores e comparar.

print("\n--- Treinando uma FLORESTA ALEATÓRIA ---")

# Criando o modelo da floresta (com 100 árvores)
floresta = RandomForestClassifier(n_estimators=100, random_state=42)

# Treinando o modelo com os mesmos dados de TREINO
floresta.fit(X_train, y_train)
```

--- Treinando uma FLORESTA ALEATÓRIA ---

```
[18]: RandomForestClassifier(random_state=42)
```

```
[19]: # Fazendo previsões nos dados de TREINO e de TESTE
pred_treino_floresta = floresta.predict(X_train)
pred_teste_floresta = floresta.predict(X_test)
```

```
[20]: # Avaliando a performance
acc_treino_floresta = accuracy_score(y_train, pred_treino_floresta)
acc_teste_floresta = accuracy_score(y_test, pred_teste_floresta)
```

```
[21]: print(f"Acurácia no TREINO: {acc_treino_floresta:.4f}␣
      ↪({classificacao(acc_treino_floresta)})")
print(f"Acurácia no TESTE: {acc_teste_floresta:.4f}␣
      ↪({classificacao(acc_teste_floresta)})")
```

Acurácia no TREINO: 1.0000 (Perfeito!)

Acurácia no TESTE: 1.0000 (Perfeito!)

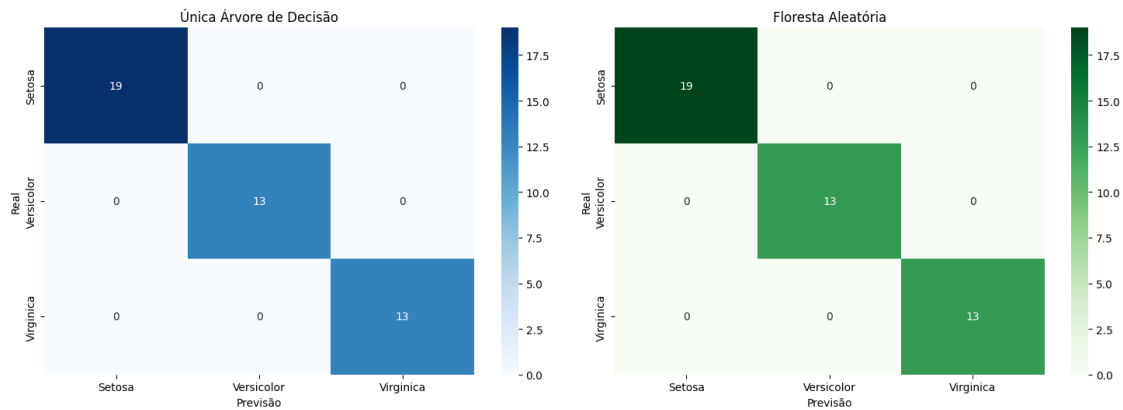
```
[22]: fig, axes = plt.subplots(1, 2, figsize=(15, 6))
fig.suptitle('Comparação de Desempenho no Conjunto de TESTE', fontsize=16)

# Matriz para a Árvore de Decisão
cm_arvore = confusion_matrix(y_test, pred_teste_arvore)
sns.heatmap(cm_arvore, annot=True, fmt='d', cmap='Blues', ax=axes[0],
            xticklabels=floresta.classes_, yticklabels=floresta.classes_)
axes[0].set_title('Única Árvore de Decisão')
axes[0].set_xlabel('Previsão')
axes[0].set_ylabel('Real')

# Matriz para a Floresta Aleatória
cm_floresta = confusion_matrix(y_test, pred_teste_floresta)
sns.heatmap(cm_floresta, annot=True, fmt='d', cmap='Greens', ax=axes[1],
            xticklabels=floresta.classes_, yticklabels=floresta.classes_)
axes[1].set_title('Floresta Aleatória')
axes[1].set_xlabel('Previsão')
axes[1].set_ylabel('Real')

plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()
```

### Comparação de Desempenho no Conjunto de TESTE



```
[23]: print("\nAnalisando as matrizes, podemos ver o número exato de erros e acertos.
        ↪")
print(f"A Árvore Única errou {cm_arvore.sum() - cm_arvore.trace()} vezes.")
print(f"A Floresta Aleatória errou {cm_floresta.sum() - cm_floresta.trace()}_
        ↪vezes.")
```

Analisando as matrizes, podemos ver o número exato de erros e acertos.

A Árvore Única errou 0 vezes.

A Floresta Aleatória errou 0 vezes.

```
[24]: # Uma grande vantagem da Floresta Aleatória é que ela pode nos dizer quais
# características (features) foram mais importantes para a tomada de decisão.

print("\n--- Importância das Features segundo a Floresta Aleatória ---")

# Criando uma série pandas para visualizar melhor
importancias = pd.Series(data=floresta.feature_importances_, index=X.columns)
importancias
```

--- Importância das Features segundo a Floresta Aleatória ---

```
[24]: sepal length (cm)    0.104105
sepal width (cm)         0.044605
petal length (cm)       0.417308
petal width (cm)        0.433982
dtype: float64
```

```
[25]: # Ordenando e pegando as 10 mais importantes
importancias_sorted = importancias.sort_values(ascending=False).head(10)
importancias_sorted
```

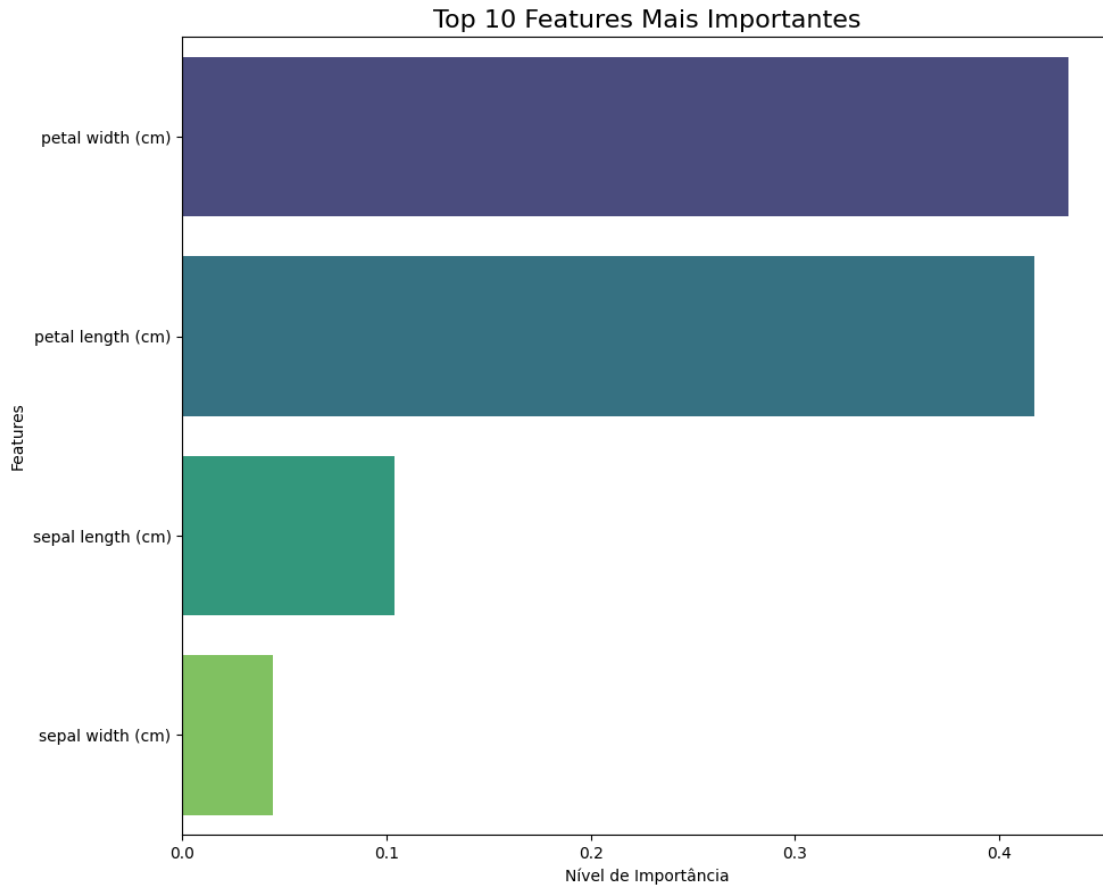
```
[25]: petal width (cm)      0.433982
      petal length (cm)   0.417308
      sepal length (cm)  0.104105
      sepal width (cm)   0.044605
      dtype: float64
```

```
[26]: # Criando o gráfico
      plt.figure(figsize=(10, 8))
      sns.barplot(x=importancias_sorted, y=importancias_sorted.index,
                  palette='viridis')
      plt.title('Top 10 Features Mais Importantes', fontsize=16)
      plt.xlabel('Nível de Importância')
      plt.ylabel('Features')
      plt.tight_layout()
      plt.show()
```

/tmp/ipykernel\_35759/1495854787.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
      sns.barplot(x=importancias_sorted, y=importancias_sorted.index,
                  palette='viridis')
```



## 1.1 Separação mais rigorosa

É muito provável que os dados do treino vazaram para o teste devido a parametros repetidos

```
[27]: # Verificação mais robusta de vazamento entre treino e teste

# 1) Índices não podem se repetir entre os conjuntos
def verificar_vazamento():
    indices_em_comum = X_train.index.intersection(X_test.index)
    assert indices_em_comum.empty, (
        f"Vazamento detectado: {len(indices_em_comum)} índices aparecem em
↪treino e teste: {list(indices_em_comum)}"
    )

# 2) Verificação adicional por conteúdo (feature + classe)
treino_check = X_train.copy()
treino_check["__target__"] = y_train.values
treino_check = treino_check.reset_index().rename(columns={"index":
↪"idx_treino"})
```

```

teste_check = X_test.copy()
teste_check["__target__"] = y_test.values
teste_check = teste_check.reset_index().rename(columns={"index": "idx_treino", "idx_teste"})

chaves = list(X.columns) + ["__target__"]
amostras_iguais = treino_check.merge(teste_check, on=chaves, how="inner")

if amostras_iguais.empty:
    print(" Verificação concluída: sem interseção por índice e sem amostras idênticas entre treino e teste.")
else:
    print(f" Atenção: existem {len(amostras_iguais)} amostras idênticas entre treino e teste (conteúdo).")
    print("Pares de índices repetidos (treino -> teste):")
    print(amostras_iguais[["idx_treino", "idx_teste"]].to_string(index=False))
    print("\nPrimeiras linhas repetidas:")
    print(amostras_iguais.head().to_string(index=False))

```

[28]: verificar\_vazamento()

Atenção: existem 1 amostras idênticas entre treino e teste (conteúdo).  
Pares de índices repetidos (treino -> teste):

| idx_treino | idx_teste |
|------------|-----------|
| 101        | 142       |

Primeiras linhas repetidas:

| idx_treino | sepal length (cm) | sepal width (cm) | petal length (cm) | petal width (cm) | __target__ | idx_teste |
|------------|-------------------|------------------|-------------------|------------------|------------|-----------|
| 101        | 5.8               | 2.7              | 5.1               |                  |            | 142       |
| 1.9        | Virginica         |                  |                   |                  |            |           |

[29]: indices = X.index.to\_series().sample(frac=1, random\_state=42).values  
ponto\_corte = int(len(indices) \* 0.7)

```

idx_treino = indices[:ponto_corte]
idx_teste = indices[ponto_corte:]

X_train = X.loc[idx_treino]
X_test = X.loc[idx_teste]
y_train = y.loc[idx_treino]
y_test = y.loc[idx_teste]
assert set(idx_treino).isdisjoint(set(idx_teste))

```

```
[30]: print("Separação manual concluída com sucesso.")
      print(f"Treino: {len(X_train)} amostras")
      print(f"Teste: {len(X_test)} amostras")
```

Separação manual concluída com sucesso.  
Treino: 105 amostras  
Teste: 45 amostras

```
[31]: # Criando o modelo da árvore
      arvore = DecisionTreeClassifier(random_state=42)

      # Treinando o modelo com os dados de TREINO
      arvore.fit(X_train, y_train)
```

```
[31]: DecisionTreeClassifier(random_state=42)
```

```
[32]: # Fazendo previsões nos dados de TREINO e de TESTE
      pred_treino_arvore = arvore.predict(X_train)
      pred_teste_arvore = arvore.predict(X_test)
```

```
[33]: # Avaliando a performance
      acc_treino_arvore = accuracy_score(y_train, pred_treino_arvore)
      acc_teste_arvore = accuracy_score(y_test, pred_teste_arvore)
```

```
[34]: acc_treino_arvore
```

```
[34]: 1.0
```

```
[35]: acc_teste_arvore
```

```
[35]: 0.9555555555555556
```

```
[36]: verificar_vazamento()
```

Verificação concluída: sem interseção por índice e sem amostras idênticas entre treino e teste.

## 2 Parte 2

```
[37]: cancer = load_breast_cancer()
      cancer
```

```
[37]: {'data': array([[1.799e+01, 1.038e+01, 1.228e+02, ..., 2.654e-01, 4.601e-01,
                    1.189e-01],
                   [2.057e+01, 1.777e+01, 1.329e+02, ..., 1.860e-01, 2.750e-01,
                    8.902e-02],
                   [1.969e+01, 2.125e+01, 1.300e+02, ..., 2.430e-01, 3.613e-01,
```

```

8.758e-02],
...,
[1.660e+01, 2.808e+01, 1.083e+02, ..., 1.418e-01, 2.218e-01,
 7.820e-02],
[2.060e+01, 2.933e+01, 1.401e+02, ..., 2.650e-01, 4.087e-01,
 1.240e-01],
[7.760e+00, 2.454e+01, 4.792e+01, ..., 0.000e+00, 2.871e-01,
 7.039e-02]], shape=(569, 30)),
'target': array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1,
1,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
0, 0, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0, 0,
1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 0, 0,
1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1,
1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 0,
0, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1,
1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 1,
1, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0, 0,
0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0,
1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 1,
1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 1, 0, 0,
0, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0,
0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0,
0, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 0,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 1, 1,
1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0,
1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1,
1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1]),
'frame': None,
'target_names': array(['malignant', 'benign'], dtype='<U9'),
'DESCR': '.. _breast_cancer_dataset:\n\nBreast cancer Wisconsin (diagnostic)
dataset\n-----\n\n**Data Set
Characteristics:**\n\nNumber of Instances: 569\n\nNumber of Attributes: 30
numeric, predictive attributes and the class\n\nAttribute Information:\n
- radius (mean of distances from center to points on the perimeter)\n
- texture (standard deviation of gray-scale values)\n
- perimeter\n
- area\n
- smoothness (local variation in radius lengths)\n
- compactness (perimeter^2 / area - 1.0)\n
- concavity (severity of concave portions of the contour)\n
- concave points (number of concave portions of the contour)\n
- symmetry\n
- fractal dimension ("coastline approximation" - 1)\n\nThe mean, standard

```



error, and "worst" or largest (mean of the three\ worst/largest values) of these features were computed for each image,\ resulting in 30 features. For instance, field 0 is Mean Radius, field\ 10 is Radius SE, field 20 is Worst Radius.\n\n - class:\n - WDBC-Malignant\n - WDBC-Benign\n\n:Summary Statistics:\n\n=====

|                                     | Min   | Max    |
|-------------------------------------|-------|--------|
| radius (mean):                      | 6.981 | 28.11  |
| texture (mean):                     | 9.71  | 39.28  |
| perimeter (mean):                   | 43.79 | 188.5  |
| area (mean):                        | 143.5 | 2501.0 |
| smoothness (mean):                  | 0.053 | 0.163  |
| compactness (mean):                 | 0.019 | 0.345  |
| concavity (mean):                   | 0.0   | 0.427  |
| concave points (mean):              | 0.0   | 0.201  |
| symmetry (mean):                    | 0.106 | 0.304  |
| fractal dimension (mean):           | 0.05  | 0.097  |
| radius (standard error):            | 0.112 | 2.873  |
| texture (standard error):           | 0.36  | 4.885  |
| perimeter (standard error):         | 0.757 | 21.98  |
| area (standard error):              | 6.802 | 542.2  |
| smoothness (standard error):        | 0.002 | 0.031  |
| compactness (standard error):       | 0.002 | 0.135  |
| concavity (standard error):         | 0.0   | 0.396  |
| concave points (standard error):    | 0.0   | 0.053  |
| symmetry (standard error):          | 0.008 | 0.079  |
| fractal dimension (standard error): | 0.001 | 0.03   |
| radius (worst):                     | 7.93  | 36.04  |
| texture (worst):                    | 12.02 | 49.54  |
| perimeter (worst):                  | 50.41 | 251.2  |
| area (worst):                       | 185.2 | 4254.0 |
| smoothness (worst):                 | 0.071 | 0.223  |
| compactness (worst):                | 0.027 | 1.058  |
| concavity (worst):                  | 0.0   | 1.252  |
| concave points (worst):             | 0.0   | 0.291  |
| symmetry (worst):                   | 0.156 | 0.664  |
| fractal dimension (worst):          | 0.055 | 0.208  |

Missing Attribute Values: None\n\n:Class Distribution: 212 - Malignant, 357 - Benign\n\n:Creator: Dr. William H. Wolberg, W. Nick Street, Olvi L. Mangasarian\n\n:Donor: Nick Street\n\n:Date: November, 1995\n\nThis is a copy of UCI ML Breast Cancer Wisconsin (Diagnostic) datasets.\n<https://goo.gl/U2Uwz2>\n\nFeatures are computed from a digitized image of a fine needle\ aspirate (FNA) of a breast mass. They describe\n characteristics of the cell nuclei present in the image.\n\nSeparating plane described above was obtained using\n Multisurface Method-Tree (MSM-T) [K. P. Bennett, "Decision Tree\n Construction Via Linear Programming." Proceedings of the 4th\n Midwest Artificial Intelligence and Cognitive Science Society,\n pp. 97-101, 1992], a classification method which uses linear\n programming to construct a decision tree. Relevant features\n were selected using an exhaustive search in the space of 1-4\n features and 1-3 separating planes.\n\nThe actual linear program used to obtain the separating plane\n in the 3-dimensional space is that described in:\n [K. P. Bennett and O. L. Mangasarian: "Robust Linear\n Programming Discrimination of Two Linearly Inseparable Sets",\n Optimization Methods and Software 1, 1992, 23-34].\n\nThis database is also available through the UW CS ftp server:\n ftp ftp.cs.wisc.edu\ncd math-prog/cpo-dataset/machine-learn/WDBC/\n\n.. dropdown:: References\n\n - W.N. Street, W.H. Wolberg and O.L. Mangasarian. Nuclear feature extraction\n for

```
breast tumor diagnosis. IS&T/SPIE 1993 International Symposium on\n
Electronic Imaging: Science and Technology, volume 1905, pages 861-870,\n    San
Jose, CA, 1993.\n - O.L. Mangasarian, W.N. Street and W.H. Wolberg. Breast
cancer diagnosis and\n    prognosis via linear programming. Operations Research,
43(4), pages 570-577,\n    July-August 1995.\n - W.H. Wolberg, W.N. Street, and
O.L. Mangasarian. Machine learning techniques\n    to diagnose breast cancer
from fine-needle aspirates. Cancer Letters 77 (1994)\n    163-171.\n',
'feature_names': array(['mean radius', 'mean texture', 'mean perimeter', 'mean
area',
    'mean smoothness', 'mean compactness', 'mean concavity',
    'mean concave points', 'mean symmetry', 'mean fractal dimension',
    'radius error', 'texture error', 'perimeter error', 'area error',
    'smoothness error', 'compactness error', 'concavity error',
    'concave points error', 'symmetry error',
    'fractal dimension error', 'worst radius', 'worst texture',
    'worst perimeter', 'worst area', 'worst smoothness',
    'worst compactness', 'worst concavity', 'worst concave points',
    'worst symmetry', 'worst fractal dimension'], dtype='<U23'),
'filename': 'breast_cancer.csv',
'data_module': 'sklearn.datasets.data'}
```

```
[38]: cancer.feature_names
```

```
[38]: array(['mean radius', 'mean texture', 'mean perimeter', 'mean area',
    'mean smoothness', 'mean compactness', 'mean concavity',
    'mean concave points', 'mean symmetry', 'mean fractal dimension',
    'radius error', 'texture error', 'perimeter error', 'area error',
    'smoothness error', 'compactness error', 'concavity error',
    'concave points error', 'symmetry error',
    'fractal dimension error', 'worst radius', 'worst texture',
    'worst perimeter', 'worst area', 'worst smoothness',
    'worst compactness', 'worst concavity', 'worst concave points',
    'worst symmetry', 'worst fractal dimension'], dtype='<U23')
```

```
[39]: X = pd.DataFrame(cancer.data, columns=cancer.feature_names)
X
```

```
[39]:
```

|     | mean radius | mean texture | mean perimeter | mean area | mean smoothness | \ |
|-----|-------------|--------------|----------------|-----------|-----------------|---|
| 0   | 17.99       | 10.38        | 122.80         | 1001.0    | 0.11840         |   |
| 1   | 20.57       | 17.77        | 132.90         | 1326.0    | 0.08474         |   |
| 2   | 19.69       | 21.25        | 130.00         | 1203.0    | 0.10960         |   |
| 3   | 11.42       | 20.38        | 77.58          | 386.1     | 0.14250         |   |
| 4   | 20.29       | 14.34        | 135.10         | 1297.0    | 0.10030         |   |
| ..  | ...         | ...          | ...            | ...       | ...             |   |
| 564 | 21.56       | 22.39        | 142.00         | 1479.0    | 0.11100         |   |
| 565 | 20.13       | 28.25        | 131.20         | 1261.0    | 0.09780         |   |
| 566 | 16.60       | 28.08        | 108.30         | 858.1     | 0.08455         |   |

|     |       |       |        |        |         |
|-----|-------|-------|--------|--------|---------|
| 567 | 20.60 | 29.33 | 140.10 | 1265.0 | 0.11780 |
| 568 | 7.76  | 24.54 | 47.92  | 181.0  | 0.05263 |

|     | mean compactness | mean concavity | mean concave points | mean symmetry \ |
|-----|------------------|----------------|---------------------|-----------------|
| 0   | 0.27760          | 0.30010        | 0.14710             | 0.2419          |
| 1   | 0.07864          | 0.08690        | 0.07017             | 0.1812          |
| 2   | 0.15990          | 0.19740        | 0.12790             | 0.2069          |
| 3   | 0.28390          | 0.24140        | 0.10520             | 0.2597          |
| 4   | 0.13280          | 0.19800        | 0.10430             | 0.1809          |
| ..  | ...              | ...            | ...                 | ...             |
| 564 | 0.11590          | 0.24390        | 0.13890             | 0.1726          |
| 565 | 0.10340          | 0.14400        | 0.09791             | 0.1752          |
| 566 | 0.10230          | 0.09251        | 0.05302             | 0.1590          |
| 567 | 0.27700          | 0.35140        | 0.15200             | 0.2397          |
| 568 | 0.04362          | 0.00000        | 0.00000             | 0.1587          |

|     | mean fractal dimension | ... | worst radius | worst texture \ |
|-----|------------------------|-----|--------------|-----------------|
| 0   | 0.07871                | ... | 25.380       | 17.33           |
| 1   | 0.05667                | ... | 24.990       | 23.41           |
| 2   | 0.05999                | ... | 23.570       | 25.53           |
| 3   | 0.09744                | ... | 14.910       | 26.50           |
| 4   | 0.05883                | ... | 22.540       | 16.67           |
| ..  | ...                    | ... | ...          | ...             |
| 564 | 0.05623                | ... | 25.450       | 26.40           |
| 565 | 0.05533                | ... | 23.690       | 38.25           |
| 566 | 0.05648                | ... | 18.980       | 34.12           |
| 567 | 0.07016                | ... | 25.740       | 39.42           |
| 568 | 0.05884                | ... | 9.456        | 30.37           |

|     | worst perimeter | worst area | worst smoothness | worst compactness \ |
|-----|-----------------|------------|------------------|---------------------|
| 0   | 184.60          | 2019.0     | 0.16220          | 0.66560             |
| 1   | 158.80          | 1956.0     | 0.12380          | 0.18660             |
| 2   | 152.50          | 1709.0     | 0.14440          | 0.42450             |
| 3   | 98.87           | 567.7      | 0.20980          | 0.86630             |
| 4   | 152.20          | 1575.0     | 0.13740          | 0.20500             |
| ..  | ...             | ...        | ...              | ...                 |
| 564 | 166.10          | 2027.0     | 0.14100          | 0.21130             |
| 565 | 155.00          | 1731.0     | 0.11660          | 0.19220             |
| 566 | 126.70          | 1124.0     | 0.11390          | 0.30940             |
| 567 | 184.60          | 1821.0     | 0.16500          | 0.86810             |
| 568 | 59.16           | 268.6      | 0.08996          | 0.06444             |

|   | worst concavity | worst concave points | worst symmetry \ |
|---|-----------------|----------------------|------------------|
| 0 | 0.7119          | 0.2654               | 0.4601           |
| 1 | 0.2416          | 0.1860               | 0.2750           |
| 2 | 0.4504          | 0.2430               | 0.3613           |
| 3 | 0.6869          | 0.2575               | 0.6638           |

|     |        |        |        |
|-----|--------|--------|--------|
| 4   | 0.4000 | 0.1625 | 0.2364 |
| ..  | ...    | ...    | ...    |
| 564 | 0.4107 | 0.2216 | 0.2060 |
| 565 | 0.3215 | 0.1628 | 0.2572 |
| 566 | 0.3403 | 0.1418 | 0.2218 |
| 567 | 0.9387 | 0.2650 | 0.4087 |
| 568 | 0.0000 | 0.0000 | 0.2871 |

|     | worst fractal dimension |
|-----|-------------------------|
| 0   | 0.11890                 |
| 1   | 0.08902                 |
| 2   | 0.08758                 |
| 3   | 0.17300                 |
| 4   | 0.07678                 |
| ..  | ...                     |
| 564 | 0.07115                 |
| 565 | 0.06637                 |
| 566 | 0.07820                 |
| 567 | 0.12400                 |
| 568 | 0.07039                 |

[569 rows x 30 columns]

```
[40]: y = pd.Series(cancer.target)
      y
```

```
[40]: 0      0
      1      0
      2      0
      3      0
      4      0
      ..
     564      0
     565      0
     566      0
     567      0
     568      1
      Length: 569, dtype: int64
```

```
[41]: # Dando nomes mais amigáveis às classes (0: Maligno, 1: Benigno)
      y = y.map({0: 'Maligno', 1: 'Benigno'})
      y
```

```
[41]: 0      Maligno
      1      Maligno
      2      Maligno
      3      Maligno
```

```

4      Maligno
      ...
564    Maligno
565    Maligno
566    Maligno
567    Maligno
568    Benigno
Length: 569, dtype: object

```

```

[42]: # Vamos separar nossos dados em dois conjuntos:
      # - TREINO: para ensinar o modelo (70% dos dados)
      # - TESTE: para avaliar o modelo em dados que ele nunca viu (30% dos dados)
      # `random_state=42` garante que a separação seja sempre a mesma, para
      ↪reprodutibilidade.
      X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
      ↪random_state=42)

```

```

[43]: X_train

```

```

[43]:      mean radius  mean texture  mean perimeter  mean area  mean smoothness  \
149      13.740      17.91      88.12      585.0      0.07944
124      13.370      16.39      86.10      553.5      0.07115
421      14.690      13.98      98.22      656.1      0.10310
195      12.910      16.33      82.53      516.4      0.07941
545      13.620      23.23      87.19      573.2      0.09246
..      ...      ...      ...      ...      ...
71       8.888      14.64      58.79      244.0      0.09783
106      11.640      18.33      75.17      412.5      0.11420
270      14.290      16.82      90.30      632.6      0.06429
435      13.980      19.62      91.12      599.5      0.10600
102      12.180      20.52      77.22      458.7      0.08013

      mean compactness  mean concavity  mean concave points  mean symmetry  \
149      0.06376      0.02881      0.01329      0.1473
124      0.07325      0.08092      0.02800      0.1422
421      0.18360      0.14500      0.06300      0.2086
195      0.05366      0.03873      0.02377      0.1829
545      0.06747      0.02974      0.02443      0.1664
..      ...      ...      ...      ...
71      0.15310      0.08606      0.02872      0.1902
106      0.10170      0.07070      0.03485      0.1801
270      0.02675      0.00725      0.00625      0.1508
435      0.11330      0.11260      0.06463      0.1669
102      0.04038      0.02383      0.01770      0.1739

      mean fractal dimension  ...  worst radius  worst texture  \
149      0.05580  ...      15.340      22.46

```

|     |         |     |        |       |
|-----|---------|-----|--------|-------|
| 124 | 0.05823 | ... | 14.260 | 22.75 |
| 421 | 0.07406 | ... | 16.460 | 18.34 |
| 195 | 0.05667 | ... | 13.880 | 22.00 |
| 545 | 0.05801 | ... | 15.350 | 29.09 |
| ..  | ...     | ... | ...    | ...   |
| 71  | 0.08980 | ... | 9.733  | 15.67 |
| 106 | 0.06520 | ... | 13.140 | 29.26 |
| 270 | 0.05376 | ... | 14.910 | 20.65 |
| 435 | 0.06544 | ... | 17.040 | 30.80 |
| 102 | 0.05677 | ... | 13.340 | 32.84 |

|     | worst perimeter | worst area | worst smoothness | worst compactness \ |
|-----|-----------------|------------|------------------|---------------------|
| 149 | 97.19           | 725.9      | 0.09711          | 0.18240             |
| 124 | 91.99           | 632.1      | 0.10250          | 0.25310             |
| 421 | 114.10          | 809.2      | 0.13120          | 0.36350             |
| 195 | 90.81           | 600.6      | 0.10970          | 0.15060             |
| 545 | 97.58           | 729.8      | 0.12160          | 0.15170             |
| ..  | ...             | ...        | ...              | ...                 |
| 71  | 62.56           | 284.4      | 0.12070          | 0.24360             |
| 106 | 85.51           | 521.7      | 0.16880          | 0.26600             |
| 270 | 94.44           | 684.6      | 0.08567          | 0.05036             |
| 435 | 113.90          | 869.3      | 0.16130          | 0.35680             |
| 102 | 84.58           | 547.8      | 0.11230          | 0.08862             |

|     | worst concavity | worst concave points | worst symmetry \ |
|-----|-----------------|----------------------|------------------|
| 149 | 0.15640         | 0.06019              | 0.2350           |
| 124 | 0.33080         | 0.08978              | 0.2048           |
| 421 | 0.32190         | 0.11080              | 0.2827           |
| 195 | 0.17640         | 0.08235              | 0.3024           |
| 545 | 0.10490         | 0.07174              | 0.2642           |
| ..  | ...             | ...                  | ...              |
| 71  | 0.14340         | 0.04786              | 0.2254           |
| 106 | 0.28730         | 0.12180              | 0.2806           |
| 270 | 0.03866         | 0.03333              | 0.2458           |
| 435 | 0.40690         | 0.18270              | 0.3179           |
| 102 | 0.11450         | 0.07431              | 0.2694           |

|     | worst fractal dimension |
|-----|-------------------------|
| 149 | 0.07014                 |
| 124 | 0.07628                 |
| 421 | 0.09208                 |
| 195 | 0.06949                 |
| 545 | 0.06953                 |
| ..  | ...                     |
| 71  | 0.10840                 |
| 106 | 0.09097                 |
| 270 | 0.06120                 |

```

435          0.10550
102          0.06878

```

```
[398 rows x 30 columns]
```

```
[44]: X_test
```

```
[44]:
```

|     | mean radius | mean texture | mean perimeter | mean area | mean smoothness | \ |
|-----|-------------|--------------|----------------|-----------|-----------------|---|
| 204 | 12.470      | 18.60        | 81.09          | 481.9     | 0.09965         |   |
| 70  | 18.940      | 21.31        | 123.60         | 1130.0    | 0.09009         |   |
| 131 | 15.460      | 19.48        | 101.70         | 748.9     | 0.10920         |   |
| 431 | 12.400      | 17.68        | 81.47          | 467.8     | 0.10540         |   |
| 540 | 11.540      | 14.44        | 74.65          | 402.9     | 0.09984         |   |
| ..  | ...         | ...          | ...            | ...       | ...             |   |
| 69  | 12.780      | 16.49        | 81.37          | 502.5     | 0.09831         |   |
| 542 | 14.740      | 25.42        | 94.70          | 668.6     | 0.08275         |   |
| 176 | 9.904       | 18.06        | 64.60          | 302.4     | 0.09699         |   |
| 501 | 13.820      | 24.49        | 92.33          | 595.9     | 0.11620         |   |
| 247 | 12.890      | 14.11        | 84.95          | 512.2     | 0.08760         |   |

|     | mean compactness | mean concavity | mean concave points | mean symmetry | \ |
|-----|------------------|----------------|---------------------|---------------|---|
| 204 | 0.10580          | 0.08005        | 0.03821             | 0.1925        |   |
| 70  | 0.10290          | 0.10800        | 0.07951             | 0.1582        |   |
| 131 | 0.12230          | 0.14660        | 0.08087             | 0.1931        |   |
| 431 | 0.13160          | 0.07741        | 0.02799             | 0.1811        |   |
| 540 | 0.11200          | 0.06737        | 0.02594             | 0.1818        |   |
| ..  | ...              | ...            | ...                 | ...           |   |
| 69  | 0.05234          | 0.03653        | 0.02864             | 0.1590        |   |
| 542 | 0.07214          | 0.04105        | 0.03027             | 0.1840        |   |
| 176 | 0.12940          | 0.13070        | 0.03716             | 0.1669        |   |
| 501 | 0.16810          | 0.13570        | 0.06759             | 0.2275        |   |
| 247 | 0.13460          | 0.13740        | 0.03980             | 0.1596        |   |

|     | mean fractal dimension | ... | worst radius | worst texture | \ |
|-----|------------------------|-----|--------------|---------------|---|
| 204 | 0.06373                | ... | 14.97        | 24.64         |   |
| 70  | 0.05461                | ... | 24.86        | 26.58         |   |
| 131 | 0.05796                | ... | 19.26        | 26.00         |   |
| 431 | 0.07102                | ... | 12.88        | 22.91         |   |
| 540 | 0.06782                | ... | 12.26        | 19.68         |   |
| ..  | ...                    | ... | ...          | ...           |   |
| 69  | 0.05653                | ... | 13.46        | 19.76         |   |
| 542 | 0.05680                | ... | 16.51        | 32.29         |   |
| 176 | 0.08116                | ... | 11.26        | 24.39         |   |
| 501 | 0.07237                | ... | 16.01        | 32.94         |   |
| 247 | 0.06409                | ... | 14.39        | 17.70         |   |

```
worst perimeter  worst area  worst smoothness  worst compactness  \
```

|     |        |        |        |         |
|-----|--------|--------|--------|---------|
| 204 | 96.05  | 677.9  | 0.1426 | 0.23780 |
| 70  | 165.90 | 1866.0 | 0.1193 | 0.23360 |
| 131 | 124.90 | 1156.0 | 0.1546 | 0.23940 |
| 431 | 89.61  | 515.8  | 0.1450 | 0.26290 |
| 540 | 78.78  | 457.8  | 0.1345 | 0.21180 |
| ..  | ...    | ...    | ...    | ...     |
| 69  | 85.67  | 554.9  | 0.1296 | 0.07061 |
| 542 | 107.40 | 826.4  | 0.1060 | 0.13760 |
| 176 | 73.07  | 390.2  | 0.1301 | 0.29500 |
| 501 | 106.00 | 788.0  | 0.1794 | 0.39660 |
| 247 | 105.00 | 639.1  | 0.1254 | 0.58490 |

|     | worst concavity | worst concave points | worst symmetry \ |
|-----|-----------------|----------------------|------------------|
| 204 | 0.2671          | 0.10150              | 0.3014           |
| 70  | 0.2687          | 0.17890              | 0.2551           |
| 131 | 0.3791          | 0.15140              | 0.2837           |
| 431 | 0.2403          | 0.07370              | 0.2556           |
| 540 | 0.1797          | 0.06918              | 0.2329           |
| ..  | ...             | ...                  | ...              |
| 69  | 0.1039          | 0.05882              | 0.2383           |
| 542 | 0.1611          | 0.10950              | 0.2722           |
| 176 | 0.3486          | 0.09910              | 0.2614           |
| 501 | 0.3381          | 0.15210              | 0.3651           |
| 247 | 0.7727          | 0.15610              | 0.2639           |

|     | worst fractal dimension |
|-----|-------------------------|
| 204 | 0.08750                 |
| 70  | 0.06589                 |
| 131 | 0.08019                 |
| 431 | 0.09359                 |
| 540 | 0.08134                 |
| ..  | ...                     |
| 69  | 0.06410                 |
| 542 | 0.06956                 |
| 176 | 0.11620                 |
| 501 | 0.11830                 |
| 247 | 0.11780                 |

[171 rows x 30 columns]

```
[45]: print(f"\nDados carregados e divididos:")
print(f" - Tamanho do conjunto de treino: {len(X_train)} amostras")
print(f" - Tamanho do conjunto de teste: {len(X_test)} amostras")
```

Dados carregados e divididos:  
- Tamanho do conjunto de treino: 398 amostras



- Tamanho do conjunto de teste: 171 amostras

```
[46]: # Criando o modelo da árvore
arvore = DecisionTreeClassifier(random_state=42)

# Treinando o modelo com os dados de TREINO
arvore.fit(X_train, y_train)
```

```
[46]: DecisionTreeClassifier(random_state=42)
```

```
[47]: # Fazendo previsões nos dados de TREINO e de TESTE
pred_treino_arvore = arvore.predict(X_train)
pred_teste_arvore = arvore.predict(X_test)
```

```
[48]: # Avaliando a performance
acc_treino_arvore = accuracy_score(y_train, pred_treino_arvore)
acc_teste_arvore = accuracy_score(y_test, pred_teste_arvore)
```

```
[49]: acc_treino_arvore
```

```
[49]: 1.0
```

```
[50]: acc_teste_arvore
```

```
[50]: 0.9415204678362573
```

```
[51]: print(f"Acurácia no TREINO: {acc_treino_arvore:.4f} (Perfeito!)")
print(f"Acurácia no TESTE: {acc_teste_arvore:.4f} (Bom, mas caiu bastante)")
print("\nOVERFITTING: a árvore 'decorou' os dados de treino (100% de acerto),")
print("mas sua performance caiu em dados novos (não vistos no treinamento),\n
      ↳pois ela é muito específica.")
```

Acurácia no TREINO: 1.0000 (Perfeito!)

Acurácia no TESTE: 0.9415 (Bom, mas caiu bastante)

OVERFITTING: a árvore 'decorou' os dados de treino (100% de acerto),  
mas sua performance caiu em dados novos (não vistos no treinamento), pois ela é  
muito específica.

```
[52]: # Agora, vamos treinar uma floresta com 100 árvores e comparar.

print("\n--- Treinando uma FLORESTA ALEATÓRIA ---")

# Criando o modelo da floresta (com 100 árvores)
floresta = RandomForestClassifier(n_estimators=100, random_state=42)

# Treinando o modelo com os mesmos dados de TREINO
floresta.fit(X_train, y_train)
```

--- Treinando uma FLORESTA ALEATÓRIA ---

```
[52]: RandomForestClassifier(random_state=42)
```

```
[53]: # Fazendo previsões nos dados de TREINO e de TESTE
pred_treino_floresta = floresta.predict(X_train)
pred_teste_floresta = floresta.predict(X_test)
```

```
[54]: # Avaliando a performance
acc_treino_floresta = accuracy_score(y_train, pred_treino_floresta)
acc_teste_floresta = accuracy_score(y_test, pred_teste_floresta)
```

```
[55]: print(f"Acurácia no TREINO: {acc_treino_floresta:.4f} (Ainda perfeito, mas mais_
      ↪robusto)")
print(f"Acurácia no TESTE: {acc_teste_floresta:.4f} (Excelente resultado!)")
print("\nNote que a Floresta Aleatória manteve uma alta performance nos dados_
      ↪de teste.")
print("A combinação de várias árvores reduziu o overfitting e criou um modelo_
      ↪mais generalista e confiável.")
```

Acurácia no TREINO: 1.0000 (Ainda perfeito, mas mais robusto)

Acurácia no TESTE: 0.9708 (Excelente resultado!)

Note que a Floresta Aleatória manteve uma alta performance nos dados de teste. A combinação de várias árvores reduziu o overfitting e criou um modelo mais generalista e confiável.

```
[56]: # Testando diferentes valores para n_estimators
valores_n_estimators = [1, 10, 20, 50, 100, 200]
resultados_rf = []

for n in valores_n_estimators:
    modelo_rf = RandomForestClassifier(n_estimators=n, random_state=42)
    modelo_rf.fit(X_train, y_train)
    y_pred_teste = modelo_rf.predict(X_test)
    acc_teste = accuracy_score(y_test, y_pred_teste)
    resultados_rf.append({"n_estimators": n, "acuracia_teste": acc_teste})

resultados_rf_df = pd.DataFrame(resultados_rf)
print(resultados_rf_df.to_string(index=False))
```

| n_estimators | acuracia_teste |
|--------------|----------------|
| 1            | 0.947368       |
| 10           | 0.970760       |
| 20           | 0.976608       |
| 50           | 0.970760       |
| 100          | 0.970760       |

200 0.970760

```
[57]: # Testando diferentes valores para max_depth com n_estimators fixo em 100
valores_max_depth = [1, 2, 3, 5, 10, None]
resultados_max_depth = []

for profundidade in valores_max_depth:
    modelo_rf_depth = RandomForestClassifier(
        n_estimators=100,
        max_depth=profundidade,
        random_state=42
    )
    modelo_rf_depth.fit(X_train, y_train)

    pred_treino = modelo_rf_depth.predict(X_train)
    pred_teste = modelo_rf_depth.predict(X_test)

    acc_treino = accuracy_score(y_train, pred_treino)
    acc_teste = accuracy_score(y_test, pred_teste)

    resultados_max_depth.append({
        "max_depth": profundidade,
        "acuracia_treino": acc_treino,
        "acuracia_teste": acc_teste
    })

resultados_max_depth_df = pd.DataFrame(resultados_max_depth)
print(resultados_max_depth_df.to_string(index=False))
```

| max_depth | acuracia_treino | acuracia_teste |
|-----------|-----------------|----------------|
| 1.0       | 0.922111        | 0.959064       |
| 2.0       | 0.964824        | 0.953216       |
| 3.0       | 0.977387        | 0.970760       |
| 5.0       | 0.992462        | 0.964912       |
| 10.0      | 1.000000        | 0.970760       |
| NaN       | 1.000000        | 0.970760       |

### 3 Questões

#### 3.0.1 1) A única Árvore de Decisão sofreu overfitting neste novo dataset? Compare as acurácias de treino e teste para justificar sua resposta.

Analisando o dataset Iris, a Árvore de Decisão obteve uma acurácia de 100% no conjunto de treino e também de 1.0 no conjunto de teste. Inicialmente, houve a suspeita de que esse resultado perfeito fosse causado por vazamento de dados entre os conjuntos. No entanto, a verificação por conteúdo confirmou que havia apenas 1 amostra idêntica entre o treino e o teste. Isso indica que o modelo não “trapaceou” decorando dados repetidos; ele obteve acurácia máxima porque o dataset Iris tem classes muito bem delineadas e é mais simples de ser classificado, assim sendo, podemos afirmar

que não ocorreu Overfitting.

**3.0.2 2) A Floresta Aleatória teve um desempenho superior ao da única árvore no conjunto de teste?**

Não, o desempenho foi igual. No contexto do dataset Iris, tanto a única Árvore de Decisão quanto a Floresta Aleatória alcançaram uma acurácia perfeita de 100% no conjunto de teste. Isso se deve ao fato do modelo já ter conseguido atingir o máximo de desempenho possível com 1 única árvore, assim sendo, não havia espaço para a floresta melhorar.

**3.0.3 3) De acordo com o gráfico gerado, qual foi a característica mais importante que o modelo usou para classificar as espécies de flores?**

A mais importante foi a petal width (cm). Ela obteve um nível de importância de aproximadamente 0.433982, já segunda característica mais utilizada pelo modelo foi a petal length (cm) com 0.417308.

**3.0.4 4) O que acontece com a acurácia à medida que aumentamos o número de árvores na floresta? A performance melhora indefinidamente ou chega a um ponto em que o ganho é muito pequeno?**

A medida que o número de árvores aumenta, a acurácia também melhora, porém depois de 20 árvores, o aumento no número de árvores na floresta para de influenciar na melhoria da acurácia uma vez que ela já estava estabilizada.

**3.0.5 5) Como o parâmetro max\_depth afeta o overfitting? Descreva o que aconteceu com a acurácia de treino e a de teste à medida que a profundidade das árvores aumentou.**

Quando a árvore cresce muito, ela acaba decorando o treino em vez de aprender sobre ele, se ela for muito curta porém, ela não consegue “lembrar” dos dados. No caso da acurácia do treino, como ela tem as respostas, assim que a árvore se tornou grande o bastante para armazenar todo o conjunto de dados, o processo caiu para 100% de precisão, já que ela já tinha todas as respostas. Quando vemos a de teste, quando a árvore era curta, ela melhorava a medida que aumentávamos o seu tamanho, porém a partir de 3, ela começa a tentar decorrar os valores em vez de aprender e portanto isso prejudica o resultado.